

1 chapter8: 回溯法

1.1 回溯法基本原理

一种更好的搜索算法，将搜索空间尽量减小，分支限界法也是这样的原理。

回溯法求解分 3 个步骤：

1. 定义一个解空间，他包含问题的解

问题的解一般可表示为向量（也称元组）形式 $X = (x_1, x_2, \dots, x_n)$ ，其中分量 x_i 的取值范围为有穷且可枚举的集合 S_i 。

每个 x_i 的所有可能取值的组合构成的集合，称为问题的解空间。其中每一个取值组合对应问题实例的一个可能解。（注意好组合问题与排序问题的不同）

对 0/1 背包问题，对 n 个物品，定义解的形式为 n 元组 $X = (x_1, x_2, \dots, x_n)$ ，其中每个分量 x_i 的取值范围 $S_i = \{0, 1\}$ 。所有 0 和 1 构成的 n 元组构成了问题的解空间，由 2^n 种可能的解组成。

例如，当 $n=3$ 时，0/1 背包问题的解空间如下： $(0,0,0), (0,0,1), (0,1,0), (0,1,1), (1,0,0), (1,0,1), (1,1,0), (1,1,1)$

解空间树通常有两种类型：子集树和排序树。当所给的问题是从 n 个元素的集合 S 中找出满足某种性质的子集时，相应的解空间树称为子集树。当所给的问题是确定 n 个元素满足某种性质的排列时，相应的解空间树称为排列树。

- 活结点: 如果已生成一个结点而它的儿子结点还没有全部生成，则这个结点叫做活结点; 当前正在生成其儿子结点的活结点叫 E-结点 (正在扩展的结点)。
- 死结点: 不再进一步扩展或者其儿子结点已经全部生成的已生成结点是死结点。

2. 用易于搜索的方式组织解空间

3. 深搜解空间，获得问题的解

回溯法原理：当从状态 s_i 搜索到状态 s_{i+1} 后，如果 s_{i+1} 变为死结点，则从状态 s_{i+1} 回退到 s_i ，再从 s_i 找其他可能的路径，所以回溯法体现出走不通就退回再走的思路。通常采用两种策略来避免无效的搜索，提高回溯法的搜索效率：

1. 使用约束函数在扩展顶点处剪去不满足约束的子树；
2. 如果是优化问题，用限界函数剪去不能得到最优解的子树。

设 $P(x_1, x_2, \dots, x_i)$ 为真表示向量 $\langle x_1, x_2, \dots, x_i \rangle$ 满足某个性质（例如， n 皇后问题中前 i 个皇后放置在彼此不能攻击的位置）。多米诺性质：

$$P(x_1, x_2, \dots, x_{k+1}) \rightarrow P(x_1, x_2, \dots, x_k) \quad 0 < k < n$$

要使回溯法得到正确应用，性质 P 必须满足多米诺性质。强调解的前缀继承性（回溯法的核心）而回溯法的本质就是约束穷举。

Input: 问题的输入

Output: 解向量 $x[1..n]$ 或 “no solution”

```
1 for  $k \leftarrow 1$  to  $n$  do
2   |  $x[k] \leftarrow k$  ;                                // 初始排列
3 end
4  $flag \leftarrow false$ ;
5 advance(1);
6 if  $flag$  then
7   | return  $x[1..n]$ 
8 else
9   | return “no solution”
10 end
11 Procedure advance( $i$ )
12   if  $i > n$  then
13     |  $flag \leftarrow true$  ;                          // 搜索到叶子结点, 找到可行解
14     | return
15   end
16   for  $j \leftarrow i$  to  $n$  do
17     | swap( $x[i], x[j]$ ) ;                               // 生成候选排列
18     | if legal( $i$ ) then
19       | advance( $i + 1$ ) ;                               // 递归进入下一层
20     | end
21     | swap( $x[i], x[j]$ ) ;                               // 回溯恢复状态
22   end
```

Algorithm 1: BACKTRACK (基于排列树的递归框架)

1.2 回溯法应用

1.2.1 图的三着色问题

给出一个无向图 $G = (V, E)$, 需要用三种颜色之一为 V 中的每个顶点着色 (三种颜色编号分别为 1, 2 和 3), 使得没有两个邻接的顶点着同样的颜色。

解的表示: 设图的顶点数为 n , 用 n 元组 $(c_1, c_2, \dots, c_n)$ 来表示顶点的一种着色方案, $c_i \in \{1, 2, 3\}$, $1 \leq i \leq n$ 。一个 n 个顶点的图共有 3^n 种可能的着色 (合法的和非法的)。

例如, $(1, 2, 2, 3, 1)$ 表示一个有 5 个顶点的图的一种着色方案, 一共有 3^5 种可能的着色 (合法的和非法的)。

所有着色方案的集合可以用一棵完全三叉树表示, 树中从根到叶子的每一条路径代表一种颜色指派。

```

Input: 无向图  $G = (V, E)$ , 顶点数  $n = |V|$ 
Output: 顶点 3 着色方案  $c[1..n]$  ( $c[i] \in \{1, 2, 3\}$ ) 或 "no solution"
1 for  $k \leftarrow 1$  to  $n$  do
2    $c[k] \leftarrow 0$ ; // 初始化颜色数组
3 end
4  $flag \leftarrow false$ ; // 解存在标志位
5  $k \leftarrow 1$ ; // 从第一个顶点开始
6 while  $k \geq 1$  do
7   while  $c[k] \leq 2$  do
8      $c[k] \leftarrow c[k] + 1$ ; // 尝试下一种颜色
9     if  $\forall (i, j) \in E, c[i] \neq c[j]$  then
10       $flag \leftarrow true$ 
11      break; // 找到合法解则退出循环
12    end
13    else if  $\forall i < k, (i, k) \in E \Rightarrow c[i] \neq c[k]$  then
14       $k \leftarrow k + 1$ ; // 当前是部分解则前进
15    end
16  end
17  if  $c[k] > 2$  then
18     $c[k] \leftarrow 0$ 
19     $k \leftarrow k - 1$ ; // 回溯到前一个顶点
20  end
21 end
22 if  $flag$  then
23   return  $c[1..n]$ ; // 输出合法着色方案
24 else
25   return "no solution"; // 无解提示
26 end

```

Algorithm 2: COLORITER (迭代图着色算法)

对于这两种算法的时间复杂性, 我们注意到在最坏情况下生成 $O(3^n)$ 个节点。对于每个生成的节点, 如果当前着色是合法的、部分解, 或者二者都不是, 那么需要 $O(n)$ 的工作来检查。

因此, 在最坏情况下, 全部的运行时间是 $O(n3^n)$ 。

空间复杂度是 $\Theta(n)$ 。

1.2.2 流水作业调度问题

有 n 个作业 (编号为 $1 \sim n$) 要在由两台机器 M_1 和 M_2 组成的流水线上完成加工。每个作业加工的顺序都是先在 M_1 上加工, 然后在 M_2 上加工。 M_1 和 M_2 加工作业 i 所需的时间分别为 a_i 和 b_i ($1 \leq i \leq n$)。

流水作业调度问题要求确定这 n 个作业的最优加工顺序, 使得从第一个作业在机器 M_1 上开始加工, 到最后一个作业在机器 M_2 上加工完成所需的时间最少。可以假定任何作业一旦开始加工, 就不允许被

中断，直到该作业被完成，即非优先调度。

Input: $a[1..n], b[1..n]$: n 个作业在 M_1 和 M_2 上的处理时间	
Output: $bestx[1..n]$: 最优调度排列, $bestf$: 最优调度总时间	
1	for $i \leftarrow 1$ to n do
2	$x[i] \leftarrow i$; // 初始化当前调度排列
3	$f2[i] \leftarrow 0$; // 初始化 M_2 完成时间数组
4	end
5	$bestf \leftarrow +\infty$; // 初始最优值为正无穷
6	$f1 \leftarrow 0$; // M_1 累计完成时间
7	Procedure $jobschedule(i)$
8	if $i > n$ then
9	if $f2[n] < bestf$ then
10	$bestf \leftarrow f2[n]$; // 更新最优值
11	for $j \leftarrow 1$ to n do
12	$bestx[j] \leftarrow x[j]$; // 复制解向量
13	end
14	end
15	return
16	end
17	for $j \leftarrow i$ to n do
18	$swap(x[i], x[j])$; // 生成候选排列
19	$f1 \leftarrow f1 + a[x[i]]$; // 更新 M_1 完成时间
20	$f2[i] \leftarrow \max(f1, f2[i-1]) + b[x[i]]$; // 计算 M_2 完成时间
21	if $ubound(i) < bestf$ then
22	$jobschedule(i+1)$; // 递归搜索
23	end
24	$f1 \leftarrow f1 - a[x[i]]$; // 回溯恢复状态
25	$swap(x[i], x[j])$; // 恢复排列顺序
26	end
27	Function $ubound(i)$
28	$tot-sum \leftarrow 0$; // 初始化未选择作业总时间
29	for $j \leftarrow i+1$ to n do
30	$tot-sum \leftarrow tot-sum + b[x[j]]$; // 累计 M_2 处理时间
31	end
32	return $f2[i] + tot-sum$; // 返回下界估计值
33	$jobschedule(1)$; // 从第 1 层开始搜索
34	return $bestx[1..n], bestf$

Algorithm 3: JobScheduleREC (流水作业调度递归算法)